Contents

Task Dispatcher for Multi-Resource Makeflow — Design Document

Companion to: `task_dispatcher_makeflow.md` (implementation plan). This document describes **what** is being built and **why**. Implementation detail (files, tests, PR slicing) lives in the plan.

---

1. Problem statement

We want to run Makeflow DAGs whose rules span multiple HPC resources, using radical.edge as the execution substrate. Three hard requirements shape the design:

1. **Makeflow stays the DAG engine.** Its scheduler, priorities, retry policy, and `makeflowlog`-based recovery are preserved verbatim. We do not reimplement DAG resolution.
2. **Rhapsody is the task-execution backend.** Rules run via rhapsody sessions on compute nodes, not via direct PsiJ submissions per rule.
3. **Execution is elastic.** The number of active compute allocations adapts to the workload. Operators do not pre-allocate static compute budgets.

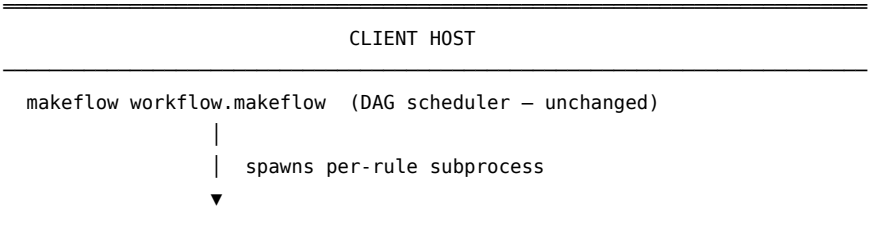
Two additional properties fell out of the iteration leading up to this doc:

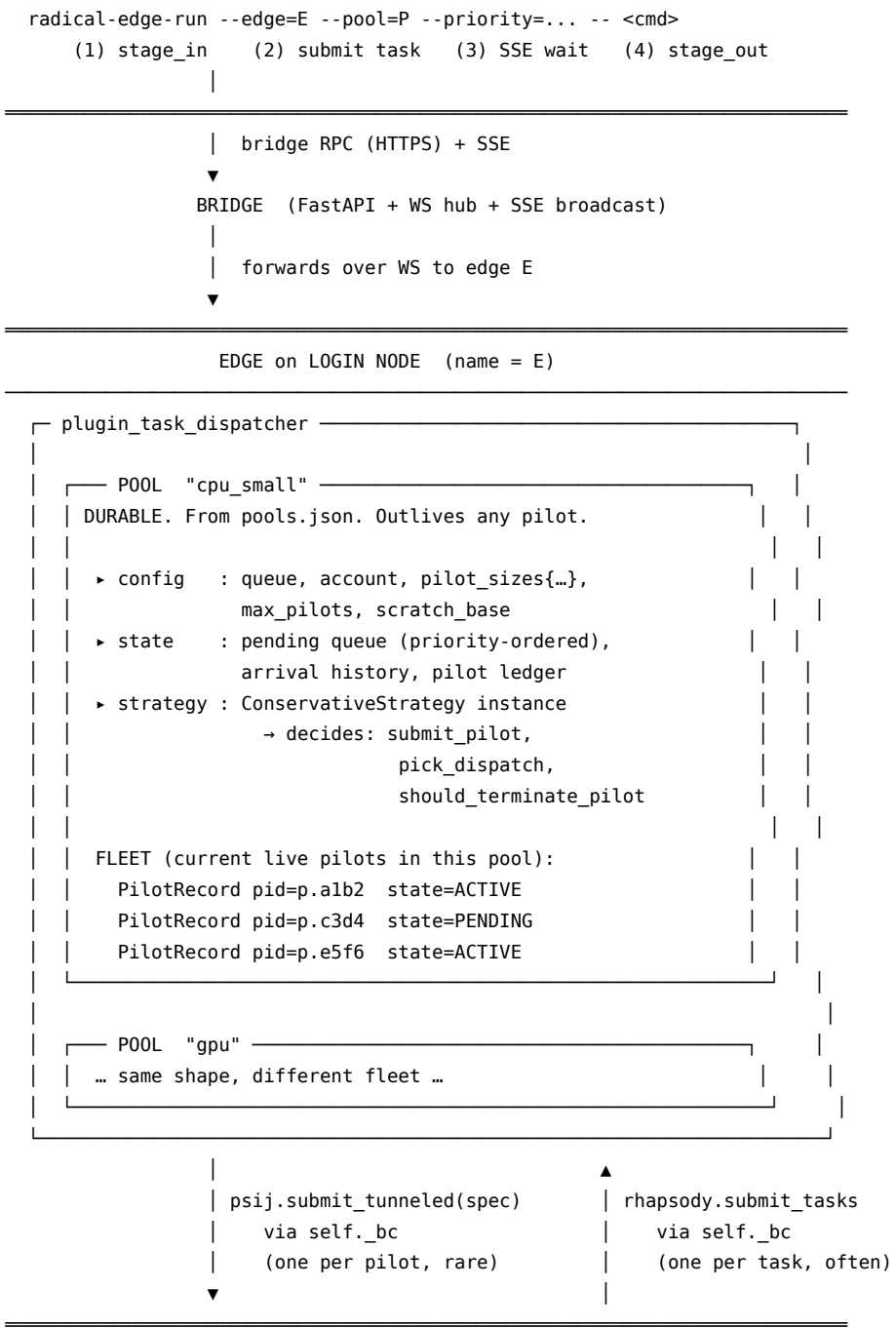
4. **Persistent, stateful coordination.** Arrival rate, fleet state, pending queues, and task→pilot assignments must survive dispatcher restarts and Makeflow-client crashes.
5. **Strategy is a research surface.** Autoscaling and task-routing policy is pluggable; different strategies are expected to coexist in the codebase and be compared experimentally.

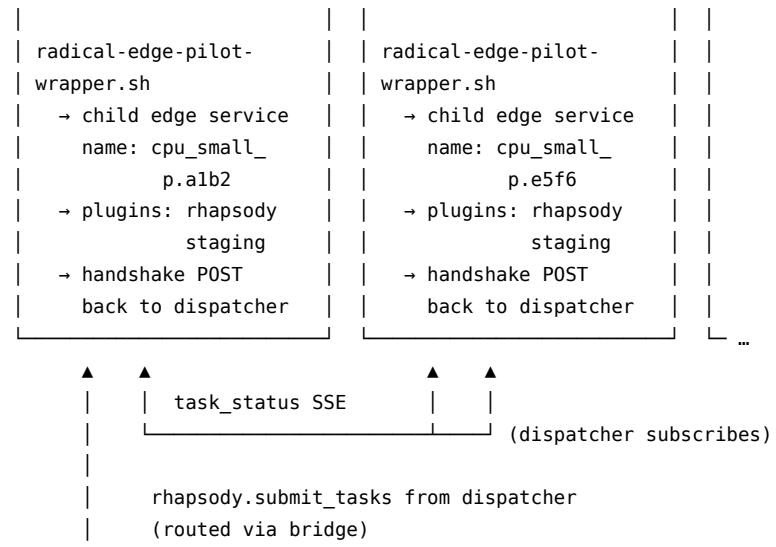
Everything in this design is in service of those five properties.

---

2. Architecture at a glance




radical-edge-run --edge=E --pool=P --priority=... -- <cmd>
(1) stage_in
(2) submit task
(3) SSE wait
(4) stage_out
bridge RPC (HTTPS) + SSE
BRIDGE (FastAPI + WS hub + SSE broadcast)
forwards over WS to edge E
EDGE on LOGIN NODE (name = E)
plugin_task_dispatcher
POOL "cpu_small"
DURABLE. From pools.json. Outlives any pilot.
▸ config : queue, account, pilot_sizes{…},
max_pilots, scratch_base
▸ state : pending queue (priority-ordered),
arrival history, pilot ledger
▸ strategy : ConservativeStrategy instance
→ decides: submit_pilot,
pick_dispatch,
should_terminate_pilot
FLEET (current live pilots in this pool):
PilotRecord pid=p.a1b2 state=ACTIVE
PilotRecord pid=p.c3d4 state=PENDING
PilotRecord pid=p.e5f6 state=ACTIVE
POOL "gpu"
… same shape, different fleet …
psij.submit_tunneled(spec)
via self._bc
(one per pilot, rare)
rhapsody.submit_tasks
via self._bc
(one per task, often)
HPC BATCH SCHEDULER (SLURM / PBS)
one batch job per pilot, waits in queue, eventually allocated
COMPUTE NODES
PILOT p.a1b2
EPHEMERAL. One SLURM job. Dies at walltime. Belongs to pool "cpu_small".
PILOT p.e5f6
EPHEMERAL. One SLURM job Dies at walltime. Belongs to pool "cpu_small".
…



---

3. Core entities

3.1 Pool

Durable scope, operator-declared, strategy-owned.

A pool is an entry in `pools.json` on a login-node edge. Its role is to be the domain of authority for a single strategy instance: one named queue and account, one menu of pilot sizes, one fleet of pilots of that menu, one pending task queue, one policy.

- Lifetime: days to weeks. A pool persists across pilot births/deaths, dispatcher restarts, and even Makeflow-client crashes (its state is on-disk).
- Cardinality: N per edge, fixed at plugin startup (no hot-reload in v1).
- Created by: operator editing `pools.json`.
- Destroyed by: operator removing the entry and restarting the edge.
- Identified by: a human-chosen string (`"cpu_small"`, `"gpu"`, ...).
- Visible to users: yes — the `POOL = "..."` directive in `.makeflow` names a pool.

A pool is the unit of resource budget (queue+account+max_pilots), the unit of policy (one strategy instance), and the unit of task grouping (one pending queue per pool; tasks do not migrate between pools).

3.2 Pilot

Ephemeral batch job on an HPC resource.

A pilot is a SLURM/PBS/LSF batch job, submitted via `plugin_psij`'s `submit_tunneled`. The job's command is `radical-edge-pilot-wrapper.sh`, which boots a child radical.edge service with the `rhapsody` and `staging` plugins loaded. That child edge connects back to the bridge and registers under a name derived from its parent pool and its own pilot id.

- Lifetime: minutes to hours, capped by walltime.
- Cardinality: 0 … `max_pilots` per pool at any instant, determined by the pool's strategy.
- Created by: strategy, via `ctx.submit_pilot(size_key)`.
- Destroyed by: walltime expiry, strategy `ctx.cancel_pilot(pid)`, or batch-system kill.
- Identified by: dispatcher-local `pilot_id = "p.<uuid8>"`; its child edge registers under `<pool>_<pilot_id>`.
- Visible to users: no. Users target pools; strategies target pilots.

A pilot owns one rhapsody session (of a specific backend) and one slice of shared-FS scratch space, reachable from both the pilot and the parent edge.

3.3 Task

One Makeflow rule, materialized at the dispatcher.

A task is the unit the dispatcher accepts and returns: a command line with declared inputs and outputs, a priority, a target pool. Its identity is stable across retries of the same Makeflow invocation.

- Lifetime: seconds to hours, bounded by the rule's command.
- Cardinality: one per invocation of `radical-edge-run`.
- Created by: `radical-edge-run` calling `td.submit_task(...)`.
- Destroyed by: terminal state (DONE/FAILED/CANCELED); record is retained for idempotency.
- Identified by: `task_id = sha1(cmd + inputs + outputs + run_id)[:16]`.
- Visible to users: indirectly — the wrapper binds task_id to one rule.

Re-submission semantics by cached state: `DONE` → return cached (crash recovery); `FAILED`/`CANCELED` → re-execute (Makeflow retry); `RUNNING` → attach to existing wait (wrapper reconnect).

3.4 Cardinality summary

```
one edge ──┬── 0…N pools              (operator-declared)
           │
           └── one plugin_task_dispatcher

one pool ──┬── one strategy instance
           ├── one pending task queue
           ├── one pilot ledger
           ├── one scratch tree
           └── 0…max_pilots live pilots

one pilot ─┬── one SLURM/PBS job
           ├── one child radical.edge service (rhapsody + staging)
           ├── one rhapsody backend (fixed at submit time)
           └── 0…capacity in-flight tasks

one task ──┬── exactly one pool (its target)
           ├── at most one pilot (once assigned)
           └── one task_id (stable across retries in a run)
```

---

4. Component responsibilities

4.1 Unchanged components

- **Makeflow** (client host). DAG resolution, priority, retries, log.
- **Bridge** (radical.edge). RPC hub, WS↔HTTPS bridging, SSE broadcast.
- **Edge service** (radical.edge). Plugin host on login node.
- **plugin_psij** (radical.edge). Used by the dispatcher to submit pilots via `submit_tunneled`.
- **plugin_rhapsody** (radical.edge). Used on each pilot's child edge to actually execute tasks.
- **plugin_staging** (radical.edge). Used on each pilot's child edge for pilot-side file access when needed. (Intra-pool file movement is avoided via shared FS.)

4.2 New components

- **plugin_task_dispatcher** (edge-side, login-node only). Owns the pool fleet, strategy, persistent state, staging scratch, task routing, and bridge SSE subscription. Single `BridgeClient` handles all outbound calls (same-edge psij, child-edge rhapsody, cross-pilot notifications).
- **DispatchStrategy** (Python ABC). Pluggable autoscaling + task-routing

  - termination policy. Ships with `ConservativeStrategy` as the v1 default.
- **radical-edge-run** (client-side CLI). Per-rule wrapper invoked by Makeflow. Stages in, submits, waits on SSE, stages out, forwards exit code.
- **radical-edge-makeflow-prep** (client-side CLI). One-shot preprocessor that rewrites a `.makeflow` file to route every rule command through `radical-edge-run`.
- **radical-edge-pilot-wrapper.sh** (HPC-side script). Runs as the pilot job's command; boots a child edge service and issues a handshake POST to the parent dispatcher.

4.3 Why this split

Keeping the dispatcher inside the login-node edge — not on the bridge — has three reasons:

- **Scheduler proximity.** Pilot submission uses psij, which is an edge-side plugin. Co-locating dispatcher with psij keeps the submission path short and allows the dispatcher to use the edge's own psij session.
- **Per-cluster policy.** Pool configuration is per-cluster (queue, account, pilot sizes are all cluster-specific). Edge-side config matches the real scope.
- **Bridge minimalism.** The bridge remains stateless relative to workflow-level scheduling; it is a transport, not a coordinator.

Cross-cluster coordination (a single dispatcher covering two clusters) is explicitly out of scope; it would require a second-tier entity above pools.

---

5. Data flow

5.1 Task-execution path (per Makeflow rule)

```
Makeflow spawns:  radical-edge-run --edge=E --pool=P --in=… --out=… -- <cmd>
                      │
                      │ 1. POST stage_in  (multipart; docstring notes bulk path)
                      ▼
Dispatcher on edge E:
  writes files to <scratch_base>/<task_id>/
  returns cwd = that path
                      │
                      │ 2. POST submit_task
                      ▼
Dispatcher:
  validates, creates TaskRecord(state=QUEUED)
  appends to log
  calls strategy.on_task_arrived(ctx, task)
  runs strategy.pick_dispatch(ctx) in a loop to drain the queue:
    each hit → _assign(task, pilot):
      calls pilot's rhapsody.submit_tasks via self._bc
      record.state = RUNNING
      emits dispatcher-level task_status SSE
                      │
                      │ 3. wrapper blocks on SSE for task_id
                      ▼
Pilot p.* executes the task via its rhapsody session
  on terminal state, pilot's rhapsody emits task_status SSE
                      │
                      ▼
Dispatcher (SSE listener on self._bc):
  matches rhapsody uid → TaskRecord
  updates state + exit_code + finished_at
```

```
  emits dispatcher-level task_status SSE
  decrements pilot.in_flight
  calls strategy.on_task_finished(ctx, task, pilot)
  runs strategy.pick_dispatch(ctx) loop (freed capacity may drain queue)
                      │
                      │ 4. wrapper wakes, pulls outputs
                      ▼
wrapper: GET stage_out for each declared output → local Makeflow workdir
wrapper: exit with task.exit_code
                      │
                      ▼
Makeflow: marks rule done/failed per exit code and output-presence check
```

5.2 Pilot-lifecycle path (strategy-driven)

```
strategy.on_tick OR on_task_arrived sees backlog exceeds capacity
  → strategy calls ctx.submit_pilot(size_key)
                      │
                      ▼
Dispatcher:
  mints pilot_id = "p." + uuid8
  mints child edge name = f"{pool}_{pilot_id}"
  builds job_spec from PilotSize:
    executable = radical-edge-pilot-wrapper.sh
    environment = { RADICAL_EDGE_PILOT_ID, ..._EDGE_NAME,
                    ..._PARENT_EDGE, ..._BRIDGE_URL,
                    ..._DISPATCHER_SID, ..._POOL,
                    ..._RHAPSODY_BACKEND (from size), ..._SCRATCH_BASE }
  creates PilotRecord(state=PENDING)
  appends to log
  calls psij.submit_tunneled(job_spec, tunnel=True) via self._bc
                      │
                      │ SLURM/PBS queues, eventually allocates, runs
                      ▼
On compute node:
  radical-edge-pilot-wrapper.sh starts radical-edge-wrapper.sh with
    --plugins rhapsody,staging --edge-name $RADICAL_EDGE_EDGE_NAME
  service registers with bridge
  inline handshake client POSTs pilot_handshake to parent dispatcher
    body: { pilot_id, child_edge, capacity, startup_time }
                      │
                      ▼
Dispatcher handshake handler:
  looks up PilotRecord by pilot_id
  records capacity, child_edge_name
  state = ACTIVE; active_at = now
  emits pilot_status SSE
  calls strategy.on_pilot_state(..., STARTING, ACTIVE)
  runs strategy.pick_dispatch(ctx) loop to drain pending queue to new pilot
                      │
                      ▼
Pilot runs tasks until:
  - walltime expires → SLURM/PBS kills the job
    → psij emits terminal state → dispatcher detects → strategy notified
  - strategy calls ctx.cancel_pilot(pid)
    → dispatcher cancels psij job → pilot terminates
```

5.3 Cross-pool file flow

Makeflow's own DAG logic handles cross-pool file dependencies without any new machinery. Each rule declares its inputs and outputs. The wrapper:

- stages declared inputs from the client's Makeflow workdir into the target pool's scratch area
- stages declared outputs from the pool's scratch back to the client's Makeflow workdir after the rule finishes

When rule A on pool `p1` produces `x.dat` and rule B on pool `p2` consumes it, rule A's wrapper pulls `x.dat` back to the client, rule B's wrapper pushes it into `p2`'s scratch. The client is a transit hub. No cross-pool primitive is required.

---

6. Protocols

6.1 Bootstrap (dispatcher → pilot)

All values flow via psij `JobSpec.environment` (not CLI args, not config file on disk). Small, atomic, no shared-FS timing dependency.

Env var	Purpose
`RADICAL_EDGE_PILOT_ID`	Pilot identity; used in handshake
`RADICAL_EDGE_EDGE_NAME`	Name under which the child edge registers
`RADICAL_EDGE_PARENT_EDGE`	Parent edge name; handshake target
`RADICAL_EDGE_DISPATCHER_SID`	Dispatcher session id
`RADICAL_EDGE_BRIDGE_URL`	Bridge URL
`RADICAL_EDGE_BRIDGE_CA`	Bridge CA bundle (optional, mTLS)
`RADICAL_EDGE_POOL`	Pool name (logging)
`RADICAL_EDGE_RHAPSODY_BACKEND`	Rhapsody backend for this pilot (from its `PilotSize`)
`RADICAL_EDGE_SCRATCH_BASE`	Shared-FS scratch path

6.2 Handshake (pilot → dispatcher)

Called once by the pilot wrapper once the child edge has registered with the bridge:

```
POST /task_dispatcher/pilot_handshake/{sid}
  body: { pilot_id, child_edge, capacity, startup_time }
```

Dispatcher: - Looks up `PilotRecord` by `pilot_id`. Unknown → 404 (rogue/late pilot). - Sets `child_edge_name`, `capacity`, `state=ACTIVE`, `active_at=now`. - Appends state transition to log. - Emits `pilot_status` SSE. - Calls `strategy.on_pilot_state(ctx, record, "STARTING", "ACTIVE")`. - Drains pending queue via `strategy.pick_dispatch` loop.

Timeout fallback: if no handshake after `max(2 * avg_startup_time, 60s)`, the dispatcher queries psij for the pilot's batch-job state. Job DONE/FAILED → `PilotRecord.state = FAILED`; any assigned tasks are re-enqueued. Job still PENDING/RUNNING → keep waiting up to walltime.

Redundant signal: `Plugin.on_topology_change` fires whenever a child edge appears or disappears. The dispatcher uses this as a secondary notice when the handshake arrives out of order; primary binding is always `pilot_id` via the handshake body.

6.3 Notifications (SSE)

Topic	Direction	Payload	Purpose
`task_status`	dispatcher → wrapper	`{task_id, state, exit_code?, ...}`	Rule-level state changes; the wrapper's wait loop consumes this
`pilot_status`	dispatcher → any SSE subscriber	`{pilot_id, state, pool, ...}`	Fleet observability; UI + research
`autoscale_decision`	dispatcher → any SSE subscriber	`{pool, action, pilot_id?, size_key?, reason?}`	Visibility into strategy decisions

Pilot-internal rhapsody `task_status` SSE is consumed by the dispatcher (not by the wrapper) and translated into dispatcher-level `task_status`. Wrappers never subscribe to pilot edges directly.

6.4 Staging (client ↔ dispatcher ↔ pilot)

Two halves:

- **Client ↔ dispatcher**: multipart upload (`stage_in`) and byte-streaming download (`stage_out`). One HTTP call per file in v1; docstrings mark the tar-stream optimization path.
- **Dispatcher ↔ pilot**: no explicit transfer. Dispatcher writes to a path under `<pool.scratch_base>/<task_id>/`; pilot's rhapsody runs with `cwd=<that path>`. Both ends see the same shared FS (Lustre / GPFS / NFS). The `sysinfo` plugin already detects shared-FS presence at edge startup.

Scratch cleanup: dispatcher TTL-sweeps task directories older than a configurable threshold (default 7 days). Non-aggressive — HPC scratch is cheap, failed runs benefit from preserved artifacts.

---

7. Strategy as research surface

7.1 Three concerns, one ABC

A strategy owns three orthogonal-enough concerns:

1. **Pilot submission** — what, when, how big.
2. **Task dispatch** — which task off the queue runs on which pilot.
3. **Pilot termination** — when to cancel (beyond walltime expiry).

Pilot replacement emerges from (1)+(3); no separate hook.

```python
class DispatchStrategy(ABC):
    def __init__(self, pool: PoolConfig, cfg: dict): ...

    # signals
    def on_task_arrived(ctx, task): ...
    def on_pilot_state(ctx, pilot, old_state, new_state): ...
    def on_task_finished(ctx, task, pilot): ...
    def on_tick(ctx): ...

    # decisions
    def pick_dispatch(ctx) -> tuple[TaskRecord, PilotRecord] | None: ...
    def should_terminate_pilot(ctx, pilot) -> bool: ...
```

`StrategyContext` exposes read-only state accessors (`pending_queue()`, `pilots()`, `arrivals_window(sec)`, `pilot_lag_history()`, `now()`, `pool`) and narrow action hooks (`submit_pilot(size_key)`, `cancel_pilot(pid)`, `drain_pilot(pid)`). The strategy never touches psij, rhapsody, or the bridge directly.

7.2 Default: ConservativeStrategy

- No eager scale-up on arrival. `pick_dispatch` routes to active pilots with capacity.
- On tick: if pending > free capacity, submit one pilot. Bounded in-flight submissions (default 2). Respect a `min_dwell_sec` between submissions.
- `pick_dispatch`: highest priority first (ties broken by arrival order); route to least-loaded active pilot.
- `should_terminate_pilot`: always False. Pilots expire at walltime.

Intended as the sensible v1 default; explicitly not the research target.

7.3 Extension points marked in code

Paired `FIXME(per-task-backend)` comments in: - `plugin_task_dispatcher.py::PluginTaskDispatcher._assign` - `task_dispatcher_strategy.py::DispatchStrategy`

Each FIXME describes the natural next hook — `strategy.pick_backend(task, pilot) -> str | None` — which would allow a strategy to override the rhapsody backend on a per-task basis instead of inheriting the pilot's default. The shared tag makes `grep -r "FIXME(per-task-backend)"` surface both sites together.

7.4 Strategy registration

Strategies load via Python entry points (`radical.edge.task_dispatcher.strategies`) or a dotted `"module:ClassName"` string in the pool config. Third-party strategies installed as separate packages show up automatically.

---

8. Configuration

8.1 Pool config (`pools.json`)

```python
@dataclass
class PilotSize:
    nodes: int
    cpus_per_node: int
    rhapsody_backend: str           # required; no cascade
    gpus_per_node: int = 0
    walltime_sec: int = 3600

@dataclass
class PoolConfig:
    name: str                           # unique per edge
    queue: str
    account: str | None
    pilot_sizes: dict[str, PilotSize]    # strategy picks by key
    default_size: str                    # key into pilot_sizes
    min_pilots: int = 0
    max_pilots: int = 4
    scratch_base: str | None = None
    strategy: str = 'conservative'
    strategy_config: dict = field(default_factory=dict)
```

- Location: `~/.radical/edge/task_dispatcher/pools.json` on each login-node edge.
- Format: JSON (stdlib; matches repo convention).
- Reload: restart-only in v1. No hot-reload.
- `rhapsody_backend` on `PilotSize` is **required, no pool-level default**. Single-backend pools repeat the string across their sizes — a deliberate verbosity trade to keep the pilot↔backend mapping explicit.

8.2 Makeflow directives

Per-rule, scoped like Makeflow's existing `CATEGORY`:

```
EDGE     = "login_node_a"
POOL     = "cpu_small"
PRIORITY = 10

out.dat: in.dat
    ./compute in.dat out.dat
```

The preprocessor rewrites the command to:

```
radical-edge-run --edge=login_node_a --pool=cpu_small --priority=10 \
                 --run-id=<sha1 of file+mtime> \
                 --in in.dat --out out.dat -- ./compute in.dat out.dat
```

Preprocessing is a one-shot step; the rewritten file is what Makeflow consumes. No runtime coupling between preprocessor and dispatcher.

---

9. Persistent state

9.1 On-disk layout

```
~/.radical/edge/task_dispatcher/
├── pools.json                      # operator-maintained config
├── state/
│   ├── <pool_name>/
│   │   ├── pilot.log               # JSONL append-only
│   │   ├── task.log                # JSONL append-only
│   │   └── snapshot.json           # periodic compaction
└── scratch/
    └── <pool_name>/<task_id>/      # rule-scoped staging dirs
```

9.2 Recovery contract

Three crash classes matter:

- **Dispatcher plugin restart.** State log is replayed into memory. Each live pilot is cross-referenced against the bridge's topology: if its child edge is still registered, the pilot is still `ACTIVE` and its in-flight tasks still routable. If the child edge is gone, the pilot is marked `FAILED` and its assigned tasks are re-enqueued.
- **Edge service restart.** Subsumes plugin restart. Pilots stay alive (they are separate processes); reconnect happens transparently via the bridge.
- **Makeflow client restart.** Makeflow's own `*.makeflowlog` plus the dispatcher's task log cover recovery. Task-id stability across retries (hash of cmd + inputs + outputs + run_id) means a re-running Makeflow will find cached `DONE` records for rules it has already completed and skip re-execution.

9.3 Idempotency boundary

On `submit_task` with a pre-existing `task_id`:

- `DONE` → return cached result (crash recovery).
- `FAILED` or `CANCELED` → overwrite record, re-execute (Makeflow retry).
- `RUNNING` or `QUEUED` → attach to existing wait (wrapper reconnect).

This keeps Makeflow's retry semantics intact while guaranteeing that a crash + restart doesn't duplicate work.

---

10. Failure modes

Event	Detection	Action
Pilot fails before handshake	Handshake timeout; psij terminal	`PilotRecord.state=FAILED`; re-enqueue assigned tasks; `on_pilot_state` fires
Pilot fails mid-task	Pilot's rhapsody emits FAILED SSE	Task marked FAILED; wrapper exits non-zero; Makeflow retries → new `task_id`? No — same task_id; dispatcher sees FAILED cached and re-executes (on the same or a different pilot)
Pilot walltime reached	psij terminal state	Running tasks reported FAILED by rhapsody-session close → Makeflow retries. Strategy may proactively `drain_pilot` before deadline
Dispatcher plugin restart	State log replay + topology reconcile	Orphan pilots with missing child edges marked FAILED; surviving pilots resume
Edge service restart	Pilots (separate processes) persist	Dispatcher reconnects to bridge; state log resumes
Makeflow-client SIGTERM to wrapper	Wrapper catches signal	Calls `cancel_task`; dispatcher cancels task on pilot; exits non-zero → Makeflow cancels run
Makeflow-client crash	Makeflow log + dispatcher log intact	Restarting Makeflow replays DAG; dispatcher returns cached results
Stage-out failure	Wrapper's HTTP GET 4xx/5xx	Exits non-zero; Makeflow treats rule as failed. Artifacts remain on the pool's scratch for diagnosis
Bridge crash / restart	Edge loses WS; wrapper loses SSE	Edge reconnects; wrapper's SSE client reconnects. Tasks in flight continue on pilots; state reconciles on bridge resume

---

11. Non-goals and deferred work

Explicitly out of scope for v1; flagged as extension points or future PRs.

- **JX workflow input.** Classic Makeflow syntax only. JX parser deferred.
- **Direct edge↔edge file transfer.** Cross-pool files transit through the client. A future edge↔edge staging primitive or a shared coordinator-hosted data plane would lift this.
- **Cross-pool coordination.** A strategy's domain is one pool. Meta- scheduling across pools (pick-least-loaded-queue policies) requires a second-tier entity above pools, future work.

- **Bridge-hosted dispatcher.** The dispatcher lives on a login-node edge for scheduler proximity and per-cluster policy locality. A bridge port exists conceptually but is deferred.
- **Hot-reload of pool config.** Restart-only.
- **Live stdout/stderr streaming during task execution.** Terminal snapshot only in v1. Adding it requires a rhapsody-side log-chunking extension and a new SSE topic — non-trivial cross-plugin change for a non-correctness feature. `TODO(streaming)` comment in the wrapper's SSE wait loop marks the insertion site.
- **Per-task rhapsody backend selection.** Backend is fixed per pilot at submit time via `PilotSize.rhapsody_backend`. Per-task override is a natural strategy extension; paired `FIXME(per-task-backend)` markers in the dispatcher's `_assign` and the strategy ABC cross-reference each other.
- **Direct pilot addressability from users.** Users target pools; the dispatcher's strategy decides pilots. No user-facing pilot handle.

---

12. Design alternatives considered and rejected

12.1 Per-edge Makeflow (`-T slurm/pbs` running on each edge)

Earlier iteration: one `makeflow` process per edge running a partitioned sub-DAG; a coordinator atop the bridge orchestrates partitioning and cross-edge staging. Rejected: duplicates DAG logic, loses Makeflow's whole-DAG visibility, introduces edge-side Makeflow install dependency, and the partition boundary becomes a fault line.

12.2 Client-owned DAG engine (rhapsody-only)

Earlier iteration: replace Makeflow entirely with a client-side DAG scheduler that directly dispatches rhapsody tasks. Rejected: loses Makeflow's battle-tested scheduler, priorities, retry policy, and transaction log. Forces us to reimplement DAG engine semantics.

12.3 Loopback HTTP for same-edge psij calls

Proposed: dispatcher uses `httpx.Client(base_url=http://localhost:<port>)` for pilot submission. Rejected: pilot submission is queue-bound (minutes to hours); bridge RTT is milliseconds. Saves nothing and doubles the error-handling surface. Single `BridgeClient` for all outbound calls.

12.4 Collapsing pool and pilot

Proposed: pilot is the pool. Rejected: collapses two entities with fundamentally different lifetimes and responsibilities. Either kills elasticity (static operator-allocated compute) or reintroduces the pool abstraction under a different name (fleet, work class, template). Pool is the entity that owns "submit another pilot like this one" and is the strategy's domain of authority; without it, strategies have no plural to reason over.

12.5 Pool-level default rhapsody backend with PilotSize override

Proposed: `PilotSize.rhapsody_backend: str | None = None` cascades to `PoolConfig.default_rhapsody_backend`. Rejected in favor of explicit required field on every `PilotSize`. Silent inheritance is a maintenance hazard when pools grow from single-backend to mixed; the verbosity of repeating the string is cheap.

---

13. Glossary

- **Edge**: a radical.edge service. In this design, always a login-node edge running the dispatcher plugin.

- **Bridge**: the central radical.edge service; RPC hub and SSE broadcast.
- **Pool**: durable scope inside the dispatcher; owns a fleet of pilots, a pending queue, and a strategy instance. Operator-declared in `pools.json`.
- **Pilot**: ephemeral SLURM/PBS batch job submitted via psij. Runs a child radical.edge service with rhapsody+staging. Belongs to one pool.
- **PilotSize**: a named entry in a pool's `pilot_sizes` map; strategy picks one by key when calling `ctx.submit_pilot(size_key)`.
- **Task**: one Makeflow rule's command, materialized at the dispatcher with stable `task_id`.
- **Strategy**: pluggable Python class implementing `DispatchStrategy`; owns the pool's scheduling decisions.
- **Wrapper**: `radical-edge-run` CLI; replaces a Makeflow rule's command and handles stage-in/submit/wait/stage-out.
- **Preprocessor**: `radical-edge-makeflow-prep` CLI; rewrites a `.makeflow` file to route every rule through the wrapper.
- **Handshake**: POST from a new pilot to the parent dispatcher carrying `{pilot_id, child_edge, capacity, startup_time}`. Binds a batch job to its dispatcher record.